PaperDatabase : une base de connaissance textuelle et vérifiable pour la lecture assistée d'articles scientifiques

Léonard Rivals Maksym Lytvynenko

28 septembre 2026

Résumé

Préparer une thèse suppose de lire, comparer et citer des dizaines d'articles. Un grand modèle de langage (LLM) peut aider, mais il travaille mal sur des PDF bruts : il en relit le texte intégral à chaque question, ne sait pas où chercher un chiffre et mélange ce que dit un article avec ce qu'il en déduit. PaperDatabase convertit chaque PDF en fichiers texte structurés (métadonnées, sections paginées, références, fiche de lecture, tableau de benchmarks) qu'un agent LLM, ici Claude Code, parcourt du résumé vers le détail. Une couche de recherche de passages, de type RAG, s'y ajoute : index lexical BM25 et index dense (bge-small-en-v1.5) fusionnés par Reciprocal Rank Fusion, avec reranking optionnel. Cette couche ne génère rien. Elle renvoie des passages identifiés, paginés et rejouables, ainsi qu'une commande qui contrôle qu'une citation, et en particulier ses nombres, figure bien dans le passage invoqué. La rédaction reste à l'agent, encadrée par des règles de sourçage. Un banc d'évaluation reproductible, sans LLM ni réseau, mesure l'extraction, la recherche, la vérification et les temps. Sur un corpus de 45 articles (3 479 passages), la recherche hybride avec reranking obtient un recall@5 de 0,80 sur un jeu de questions réservé, qui n'a servi à aucun réglage, contre 1,00 sur le jeu qui a guidé la conception, et de 0,50 en français. Passer de 28 à 45 articles, à questions constantes, fait baisser le mode hybride sans reranking de 0,80 à 0,72 sur le jeu réservé. Un second moteur, optionnel, sur GPU (Qwen3-Embedding-4B et Qwen3-Reranker-4B) retrouve les 8 questions françaises, contre 5 pour le meilleur mode sur CPU. La vérification refuse toutes les citations dont un chiffre a été modifié, qu'elle acceptait auparavant. Aucun signal simple ne s'est montré fiable pour détecter qu'une question sort du corpus. Un essai réel de réponse sourcée a consommé environ 2 500 tokens, alors qu'un seul texte intégral en représente environ 13 000 en médiane. Le même code sert une seconde base, sur la vision embarquée sur drone (73 articles, 4 525 passages), avec ses propres jeux de questions : le reranking y obtient 1,00 sur le jeu de développement et 0,95 sur le jeu réservé, et les mêmes faiblesses (tableaux, français, questions hors corpus) s'y retrouvent.

Table des matières

1 Introduction **3**

2 Principes de conception **3**

3 Architecture et pipeline **4**
3.1 Acquisition et ingestion 4
3.2 Modèle de données 5
3.3 Plusieurs bases 5
3.4 Modules 6

4 La couche de recherche (RAG) **6**
4.1 Un RAG sans génération 6
4.2 Découpage en passages 6
4.3 Deux index et une fusion 7
4.4 Commandes 8
4.5 Protocole de réponse : la skill `/paper-ask` 9
4.6 Un second moteur sur GPU et l'acquisition open access 9

5 Ce que le logiciel apporte **10**

6 Tests et évaluation **12**
6.1 Tests unitaires 12
6.2 Le banc d'évaluation `pdb eval` 12
6.3 Qualité de l'extraction 13
6.4 Recherche de passages 14
6.5 Détecter qu'une question sort du corpus 16
6.6 Vérification des citations 16
6.7 Temps d'exécution 17
6.8 Essai réel de bout en bout 17
6.9 Un second corpus : la base drone 17
6.10 Banc synthétique (`pdb synth`) 18
6.11 Menaces à la validité 19

7 Limites et perspectives **19**

8 Conclusion **20**

A Glossaire des acronymes **22**

B Lexique **23**

C Commandes **24**

1 Introduction

Contexte. Ce travail est né d'un besoin concret : préparer un projet de thèse sur la vision embarquée sur FPGA (détecteurs YOLO, autoencodeurs, apprentissage continu). Le domaine publie beaucoup de chiffres (images par seconde, GOPS, watts, ressources LUT, DSP et BRAM), mesurés dans des conditions hétérogènes. Les comparer demande de savoir d'où vient chaque valeur. Le corpus de ce cas d'usage sert de banc d'essai, mais l'outil ne dépend pas du domaine : le même code sert aujourd'hui une seconde base, sur la vision embarquée sur drone, et ce qui dépend du sujet (axes, règles de classement, tags, périmètre) est déclaré dans les fichiers de chaque base.

Problème. Un agent LLM à qui l'on donne des PDF a trois défauts :

1. *coût* : il relit des textes intégraux (environ 13 000 tokens en médiane) pour répondre à une question qui tient en trois phrases ;
2. *traçabilité* : un chiffre ou une citation ne porte pas l'endroit exact d'où il vient, et une citation reformulée passe pour textuelle ;
3. *confusion entre lu et déduit* : l'agent comble les manques avec ses connaissances générales, sans le signaler.

Approche. PaperDatabase ne cherche pas à remplacer l'agent. Il lui fournit une base où chaque information porte sa source, et des commandes déterministes qui répondent aux questions factuelles sans appel à un LLM. La génération se limite aux questions qui demandent une synthèse, et elle se fait à partir de sorties déjà sourcées.

Contributions.

- Un pipeline d'ingestion PDF → texte structuré (GROBID, repli PyMuPDF, réparation des ligatures, reconstruction des tableaux), idempotent, qui ne devine jamais une métadonnée absente (section 3).
- Un modèle de données entièrement textuel, validé à chaque réindexation, où toute valeur numérique porte sa section source et un niveau de confiance (section 3.2).
- Une couche de recherche hybride sans génération, organisée en trois niveaux de coût et dotée d'une vérification automatique des citations (section 4).
- Un banc d'évaluation reproductible, sans LLM ni réseau, qui mesure la qualité de l'extraction, de la recherche et de la vérification des citations, ainsi que les temps, avec un jeu de questions réservé qui n'a servi à régler aucun paramètre ; et une suite de tests unitaires qui s'exécute sans modèle (section 6).
- Un second moteur de recherche, optionnel, sur GPU (Qwen3), et une acquisition d'articles open access validée par l'utilisateur (section 4.6), comparés au premier sur les mêmes jeux de questions.
- Une séparation du code et des données qui permet de tenir plusieurs bases isolées, chacune avec son profil, ses règles et son banc d'évaluation, et d'échanger des articles entre elles (section 3.3) ; le banc est reproduit sur un second corpus (section 6.9).

2 Principes de conception

Le consommateur final de la base n'est pas un moteur de recherche. C'est un agent LLM qui dispose déjà d'outils de lecture de fichiers et de recherche textuelle (`grep`, `read`). Sur un corpus de quelques dizaines à quelques centaines d'articles, une arborescence bien nommée, un index et un CSV filtrable suffisent pour l'essentiel des questions, et restent lisibles, comparables d'une version à l'autre et réparables à la main. Quatre règles en découlent.

1. **Tout est du texte.** Pas de base de données ni de format binaire, à part le PDF d'origine, conservé comme preuve. L'index de recherche (section 4) est un cache dérivé, reconstructible, hors de l'arborescence versionnée.
2. **Chaque chiffre est traçable.** Toute valeur du tableau de benchmarks porte la section, la table ou la figure dont elle vient. Une valeur non rapportée par les auteurs s'écrit `n/r`, jamais une estimation.
3. **Lecture du résumé vers le détail.** L'agent lit la couche la plus haute qui répond, et ne descend que si nécessaire (tableau 1). Il ne rouvre jamais le PDF.
4. **Le schéma est codé à un seul endroit** (`lib/schema.py`) et vérifié à chaque réindexation, qui s'arrête au premier écart.

TABLE 1 – Couches de lecture, du résumé au détail.

Couche	Fichier ou commande	Sert à
1	`INDEX.md` (généré)	savoir quels articles existent, par axe, fichés ou non
2	`benchmarks.csv` via `pdb bench`	trouver un chiffre, avec sa section source
3	`papers/<id>/fiche.md`	lire l'analyse critique d'un article
4	`pdb search`, `show`, `verify`	trouver et citer les 2 ou 3 passages qui répondent
–	`fulltext.md`	texte intégral, en dernier recours

Un fichier d'instructions (`CLAUDE.md`) décrit à l'agent cet ordre de lecture et des règles de travail : ne jamais inventer de référence, écrire `n/r`, séparer ce qui est lu de ce qui est déduit, signaler les preprints et les métriques non comparables.

3 Architecture et pipeline

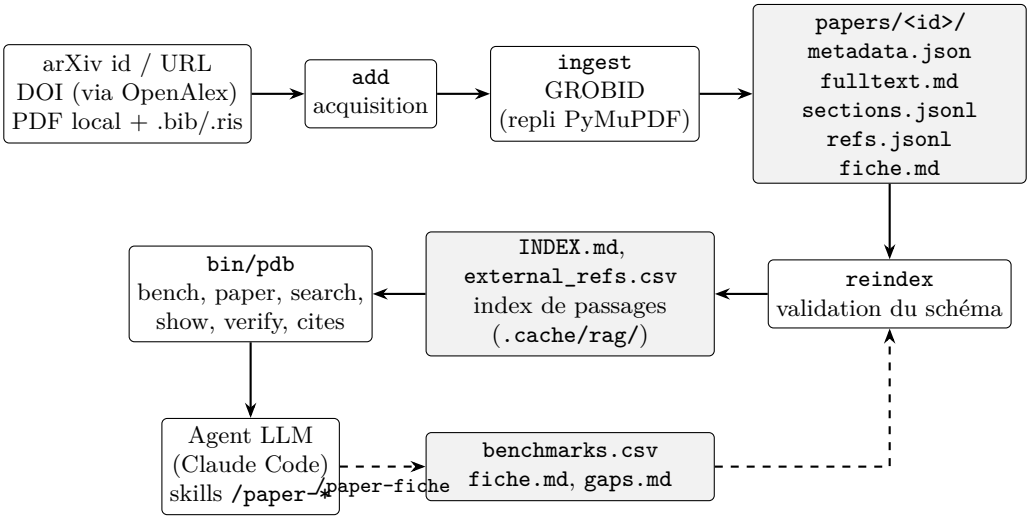


FIGURE 1 – Flux de données. Les flèches pleines sont automatiques ; les flèches en tirets passent par l'agent, qui rédige les fiches et les lignes de benchmark à partir du texte extrait.

3.1 Acquisition et ingestion

`bin/add.py` récupère un article arXiv (métadonnées par l'API, puis le PDF), résout un DOI vers sa version arXiv par OpenAlex (Priem et al., 2022), ou copie un PDF local. Les articles sous paywall se déposent à la main : l'outil ne contourne aucun accès.

`bin/ingest.py` envoie chaque PDF à GROBID (GROBID contributors, 2008–2026), qui renvoie une structure TEI (en-tête, sections, figures, bibliographie). En l'absence de GROBID, une extraction de repli par PyMuPDF découpe le texte par heuristique sur les titres numérotés, et le signale dans les métadonnées. Le pipeline applique ensuite, dans l'ordre :

— *fusion des métadonnées* par ordre de confiance : export de citation de l'éditeur, puis arXiv, puis GROBID. Un champ inconnu reste `null` ;
— *nettoyage* : en-têtes et pieds répétés, numéros de page, bibliographie sortie du corps et rangée dans `refs.jsonl` ;
— *réparation des ligatures perdues* (« classi cation » pour « classification ») : un mot n'est recollé que si le résultat est connu du corpus ou d'une liste intégrée, et que le fragment ne l'est pas ;
— *tableaux* : un tableau dont GROBID n'a gardé que la légende passe par deux replis successifs. PyMuPDF le cherche d'abord sur sa page (tableaux à filets). Docling (Auer et al., 2024) vient ensuite, sur demande (`ingest --reparse --tables docling`), avec une reconnaissance optique (OCR) en option pour les tableaux publiés en image. Un corps reconstruit est marqué « reconstruit par PyMuPDF / Docling : à vérifier », et un tableau resté vide est signalé par `pdb status` ;
— *paragraphes mal classés* : un paragraphe que GROBID a pris pour une figure est rendu au corps quand il ressemble à de la prose.

L'ingestion est idempotente, et une ré-extraction (`--force`, ou `--reparse` depuis les TEI conservés) n'écrase jamais une fiche de lecture déjà rédigée ni une classification corrigée à la main.

3.2 Modèle de données

Chaque article devient un dossier `papers/<paper_id>/`. L'identifiant est stable et lisible (`arxiv-<id>-<slug>` ou `<année>-<auteur>-<slug>`). Le texte extrait existe sous deux formes : `fulltext.md`, lisible, et `sections.jsonl`, une ligne par section avec son type (résumé, section, figure, table) et sa page. Cette seconde forme est la matière première de l'index de passages.

Le fichier `benchmarks.csv` contient une ligne par configuration rapportée : un article qui présente trois variantes produit trois lignes. Outre les colonnes de mesure (modèle, plateforme, quantification, FPS, GOPS, latence, puissance, LUT, DSP, BRAM, jeu de données, toolflow), trois colonnes servent à juger chaque valeur :

— `source_section` : section, table ou figure d'origine ;
— `confidence` : `high` (lu dans une table), `medium` (lu dans le texte) ou `low` (déduit, à revérifier) ;
— `power_scope` : portée de la puissance rapportée (`chip`, `board`, `board-dynamic`, `estimated`, `unspecified`). Deux puissances de portées différentes ne se comparent pas : une carte peut consommer plusieurs watts au repos, qu'une mesure « carte » inclut et qu'une mesure « puce » exclut.

Les fiches de lecture (`fiche.md`) suivent un gabarit fixe (problème, méthode, résultats, limites, pertinence) avec une section « statut de lecture » obligatoire qui sépare le lu du déduit. Elles sont rédigées par l'agent (commande `/paper-fiche`) et relues par l'utilisateur.

3.3 Plusieurs bases

Le même code sert plusieurs bases de connaissance, une par sujet. Le dépôt en contient deux : la base *fpga* du cas d'usage, qui sert de référence, et une base *drone* (vision embarquée sur drone : détection, accélérateurs, co-design calcul, énergie et vol), créée ensuite sur un sujet voisin (tableau 2). La racine du dépôt ne contient que le code. Chaque base est un dossier `bases/<nom>/` qui a ses articles, son `benchmarks.csv`, sa carte des lacunes, son index de passages et ses caches ; les commandes, l'environnement Python et les skills de l'agent y sont des liens vers le code.

Ce qui dépend du domaine tient dans deux fichiers par base. `base.json` déclare les axes de veille et des règles, en expressions régulières, qui classent un article dans un axe, lui attribuent des

TABLE 2 – Les deux bases au 28/09/2026.

Base	Articles	Fiches faites	Lignes de benchmark	Passages indexés
fpga (référence)	45	41	172	3 479
drone	73	73	324	4 525

tags et décident s'il entre dans le sujet (ce dernier test sert à la découverte). `profile.md` décrit en texte le thème et le périmètre (inclus, exclu) ; la skill `/base-profile` propose de l'affiner à partir des fiches, et n'écrit rien sans accord. Chaque base a aussi son `CLAUDE.md`, qui porte les règles de travail propres au domaine : pour la base drone, préciser la gamme de puissance d'un résultat, ou distinguer une détection sol → air d'une détection air → air. Les colonnes de `benchmarks.csv` restent communes à toutes les bases.

La base active se déduit du contexte : la variable `$PDB_ROOT` (fixée par `pdb --base <nom>`), sinon `$PDB_BASE`, sinon le premier dossier contenant `base.json` en remontant depuis le répertoire courant, sinon la base par défaut d'un registre local (`~/.config/pdb/bases.json`), sinon la base de référence. Deux sessions de l'agent ouvertes dans deux bases travaillent donc chacune sur la sienne.

Les bases restent séparées, mais un article peut passer de l'une à l'autre. `pdb base scan` cherche dans les autres bases les articles utiles à la base courante : ceux qu'elle cite déjà sans les avoir, ceux qui correspondent à ses règles, et les références citées des deux côtés. `pdb base import` copie un article sans le réextraire et remet sa fiche à relire, car sa pertinence dépend du sujet de la base qui le reçoit.

3.4 Modules

Toutes les commandes passent par un point d'entrée unique, `bin/pdb`. Leur sortie suit les mêmes conventions : TSV par défaut, une ligne d'en-tête sans bannière, des notes préfixées par `#`, des identifiants courts d'articles (`1612.07119`, `2022-feng`) avec leur légende en pied, et un code de sortie qui distingue « des résultats », « aucun résultat » et « erreur d'argument ».

4 La couche de recherche (RAG)

4.1 Un RAG sans génération

Dans un système de *Retrieval-Augmented Generation* (Lewis et al., 2020), le LLM reçoit, avec la question, les passages d'un corpus qui y répondent, et rédige à partir d'eux. PaperDatabase sépare les deux moitiés. Les commandes `bin/pdb` font la recherche, de façon déterministe et sans appel à un LLM. L'agent les appelle, lit leurs sorties et ne rédige que si une synthèse est nécessaire. Ce choix n'ajoute ni clé d'API ni coût : la génération est faite par l'agent déjà en service.

Trois objectifs guident la conception, par ordre de priorité : la *vérifiabilité* (tout élément renvoyé porte une source rejouable), l'*économie de tokens* (répondre sans lire de texte intégral), et l'*accès par le sens* (retrouver « continual learning » en cherchant « apprentissage incrémental », ce que `grep` ne sait pas faire).

Une sortie N0 ou N1 est souvent déjà une réponse : un tableau filtré ou trois passages sourcés suffisent à une question factuelle, et l'agent peut les transmettre sans les reformuler. Le niveau N2 n'ajoute pas d'information : il relie ce que N0 et N1 ont renvoyé, et il étiquette ses inférences comme telles.

4.2 Découpage en passages

L'unité de départ est une entrée de `sections.jsonl`, qui respecte déjà les paragraphes. Une entrée de plus de 250 mots est coupée en fenêtres de 200 mots avec 40 mots de recouvrement,

TABLE 3 – Principaux modules de `lib/`. L'ensemble est écrit en Python ; seules la recherche dense, le moteur GPU, les figures et la visualisation ont des dépendances optionnelles.

Module	Rôle
`tei.py`, `doc.py`, `pdftext.py`	lecture de la sortie GROBID, représentation commune, repli PyMuPDF
`ligatures.py`, `tables.py`, `docling_tables.py`	réparation des ligatures, reconstruction des tableaux (PyMuPDF, puis Docling)
`citation.py`, `openalex.py`	exports `.bib`/`.ris`, client OpenAlex avec cache
`schema.py`	validateurs du modèle de données, seul endroit où il est codé
`refs.py`	rapprochement des références (arXiv, DOI, titre normalisé), citations internes
`query.py`	requêtes structurées sans LLM (`bench`, `paper`, `cites`)
`rag.py`	passages, BM25, index dense, fusion, reranking, vérification des citations
`rag_qwen.py`, `rag_models.py`	second moteur sur GPU : FTS5, Qwen3-Embedding-4B, Qwen3-Reranker-4B
`rag_external.py`	acquisition open access par étapes (OpenAlex, arXiv), import après accord
`evaluation.py`	banc d'évaluation (`pdb eval`) : extraction, recherche, `verify`, temps
`status.py`	état de la base et prochaines actions
`base.py`, `registry.py`	base active, lecture de `base.json`, classement selon ses règles ; registre des bases
`bases.py`, `xbase.py`, `vizbases.py`	création et contrôle des bases, scan et import entre bases, onglet Bases de la visualisation
`plotdata.py`, `plotfig.py`	figures statiques (`pdb plot`) : benchmarks, évaluation, corpus
`graph.py`, `lineage.py`, `vizfig.py`, `embed.py`	visualisation 3D et généalogie (outil secondaire)

parce que les modèles d'embeddings tronquent au-delà de 256 à 512 tokens. Chaque passage reçoit un identifiant rejouable, `<section>.<k>` (par exemple `1612.07119#019.3`), et garde son article, sa section, sa page et l'empreinte SHA-1 de son texte. Le texte indexé est préfixé du titre de section, ce qui donne du contexte sans modifier le texte renvoyé. Le corpus actuel compte 3 479 passages.

4.3 Deux index et une fusion

Index lexical. Un BM25 (Robertson and Zaragoza, 2009), écrit à la main ($k_1 = 1{,}5$, $b = 0{,}75$, jetons alphanumériques en minuscules), retrouve les termes exacts : noms de cartes (`ZCU104`), de modèles (`YOLOv5n`), sigles et nombres.

Index dense. Le modèle `BAAI/bge-small-en-v1.5` (Xiao et al., 2024) (384 dimensions), chargé par sentence-transformers (Reimers and Gurevych, 2019), encode chaque passage en un vecteur normalisé. Il retrouve les reformulations et, dans une certaine mesure, les requêtes dans une autre langue. La requête reçoit le préfixe d'instruction recommandé par le modèle ; les passages, non.

Fusion. Les deux classements sont fusionnés par Reciprocal Rank Fusion (Cormack et al., 2009) :

$$\text{score}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{60 + \text{rang}_i(d)}. \tag{1}$$

Cette méthode ne dépend pas de l'échelle des scores, ne demande aucun réglage et reste déterministe. En cas d'égalité, l'identifiant le plus petit passe en premier : la même requête sur le même index

TABLE 4 – Trois niveaux de requête, du moins cher au plus cher. On ne monte d'un niveau que si le précédent ne suffit pas. Coûts relevés sur le corpus actuel.

Niveau	Source	LLM	Coût typique
N0 – requête structurée	`benchmarks.csv`, métadonnées	non	100–500 tokens
N1 – recherche de passages	index de passages	non	400–800 tokens, +300 par passage ouvert
N2 – synthèse	sorties de N0 et N1 seulement	oui	celui de la rédaction

donne toujours le même classement.

Pertinence absolue. Le score RRF est un rang, pas une mesure de pertinence : `search` renvoie toujours k passages, même quand la base ne traite pas le sujet. La sortie affiche le cosinus entre la requête et chaque passage. Une première version avertissait (`# pertinence faible`) quand le meilleur cosinus passait sous un seuil calibré à la main sur quelques requêtes. Le banc d'évaluation a montré que cette alerte ne se déclenchait jamais, et qu'aucun seuil, ni sur le cosinus ni sur le score d'un cross-encoder, ne séparait les questions hors corpus des autres (section 6.5). Elle a donc été retirée, parce qu'elle rassurait à tort : c'est l'agent qui juge, à la lecture des extraits, si la base répond, et qui le dit sinon au lieu de combler le vide.

Reranking (option). `--rerank` reclasse les 30 premiers résultats avec le cross-encoder `ms-marco-MiniLM-L-6-` un MiniLM (Wang et al., 2020) entraîné sur MS MARCO (Nguyen et al., 2016) selon l'approche de Nogueira and Cho (2019). Il coûte environ 1,4 s par recherche à chaud. Il n'est pas activé par défaut, pour les raisons données en section 6.4.

Stockage. Les vecteurs sont rangés dans `.cache/rag/<modèle>.npz`, indexés par la clé `chunk_id:sha1`. La mise à jour est incrémentale : seuls les passages nouveaux ou modifiés sont encodés (environ 3 min 40 pour une construction complète sur CPU, soit 16 passages par seconde ; 0,2 s sans changement). Les passages et le BM25 sont recalculés à chaque commande en moins d'une seconde. Sans sentence-transformers, ou si l'index est périmé, `search` passe en lexical seul et le signale. À cette échelle, une recherche exhaustive en numpy prend quelques millisecondes : une base vectorielle dédiée n'apporterait rien avant de l'ordre de 10^5 passages (estimation, non mesurée).

4.4 Commandes

`pdb bench` filtre `benchmarks.csv` (sous-chaînes, bornes numériques, colonnes renseignées, tri, agrégation par `--count`). Les colonnes de source (`source_section`, `confidence`, `power_scope`) restent affichées même si on ne les demande pas, et une ligne de confiance basse est préfixée par `!`.

```
$ bin/pdb bench --platform kv260 --cols model,quantization,fps,power_w
paper       model                quantization  fps  power_w  power_scope  source_section  confidence
2507.18174  YOLOv5 (variante n/r)  INT8          9    3.5      unspecified  Table I         high
# ids: 2507.18174=arxiv-2507.18174-real-time-object-detection-classification
```

`pdb search` renvoie une ligne par passage : identifiant, section, page, cosinus et un extrait d'environ 160 caractères centré sur le terme le plus rare de la requête (termes rares entre guillemets). Sortie réelle, extraits raccourcis :

```
$ bin/pdb search "power measured at the wall versus the FPGA rails" -k 3
chunk              section                      page  cos   snippet
1612.07119#022.0    5. EVALUATION / 5.2.2. ...      8     0.78  ...both the «wall» «power» and «FPGA» «power
    ↪ » readings...
2025-hozhabr#051.1  V. RESULTS ANALYSIS ... / B. ...  31    0.77  ...computational «power», or throughput, «
    ↪ measured» in...
1612.07119#019.3    5. EVALUATION / 5.1. ...        7     0.73  ...PMBus interface to monitor the «FPGA» «
    ↪ power» supply «rails»...
```

`pdb show` affiche le texte exact d'un passage, avec son en-tête (section, page, début du SHA-1), et, sur demande, ses voisins. `pdb verify` contrôle qu'une citation figure dans le passage invoqué. Les deux textes sont normalisés (ligatures, casse, césures, tout ce qui n'est ni lettre ni chiffre), puis comparés par recherche exacte ou, à défaut, par la meilleure fenêtre de `difflib`. Le verdict est l'un des cinq suivants :

- `OK` : la citation figure telle quelle, après normalisation ;
- `APPROX` : ressemblance d'au moins 0,85 (paraphrase proche) ;
- `NOMBRES` : ressemblance d'au moins 0,85, mais les nombres de la citation ne sont pas ceux du passage ;
- `ABSENT` : ressemblance inférieure à 0,85 ;
- `MODIFIÉ` : le passage a changé depuis que son SHA-1 a été relevé.

Seuls `OK` et `APPROX` donnent un code de sortie nul. Le statut `NOMBRES` est issu du banc d'évaluation. Dans une phrase longue, un chiffre changé laisse la ressemblance au-dessus du seuil, et la première version acceptait ainsi toutes les citations dont un chiffre avait été modifié (section 6.6). Le contrôle ajouté exige que les nombres de la citation forment une suite contiguë des nombres du passage. Chacun doit y figurer avec ses mots voisins : « Section 3 reviews » est refusé même si le passage contient un 3 ailleurs. Comme la normalisation retire la ponctuation, c'est aussi ce contrôle qui distingue « 94.9 % » de « 949 % ».

Une citation absente du passage indiqué mais présente ailleurs dans le même article est signalée avec l'identifiant où elle a été trouvée, et la commande sort en erreur : c'est la source qui était fausse.

```
$ bin/pdb verify 1612.07119#019.3 "P-wall using a wall-power meter for the whole board power consumption"
APPROX  0.95  1612.07119#019.3
```

Enfin, `pdb paper` donne l'identité d'un article (auteurs, venue, preprint ou non, statut de la fiche), et `pdb cites` / `cited-by` parcourent le graphe des citations internes au corpus, reconstruit en rapprochant les références par identifiant arXiv, DOI ou titre normalisé.

4.5 Protocole de réponse : la skill `/paper-ask`

La skill `/paper-ask` fixe à l'agent un déroulé, et ne l'autorise qu'à utiliser les commandes `bin/pdb` et la lecture de fichiers :

1. question chiffrée : `pdb bench` d'abord (N0) ;
2. sinon, ou en complément : `pdb search --rerank` avec une ou deux formulations en anglais ; si aucun extrait ne répond, dire que la base ne traite probablement pas le sujet, au lieu de combler le manque ;
3. `pdb show` sur 2 ou 3 passages au plus, jamais sur le texte intégral ;
4. réponse avec citations `[chunk_id]`, et `pdb verify` sur chaque citation textuelle : une citation `ABSENT` est retirée, une citation `NOMBRES` est corrigée d'après le texte exact ou retirée, un identifiant faux est corrigé, et une citation `APPROX` n'est gardée que présentée comme une paraphrase ;
5. structure imposée : **Lu** (sourcé), **Déduit**, **Non trouvé dans la base**, avec signalement des valeurs de confiance basse, des preprints et des portées de puissance hétérogènes.

4.6 Un second moteur sur GPU et l'acquisition open access

Moteur. `bin/pdb rag` propose une seconde chaîne de recherche, indépendante de la première et réservée à une machine équipée d'un GPU (mesures faites sur une RTX 5090 de 32 Go) : un index lexical SQLite FTS5, un index dense `Qwen3-Embedding-4B`, la même fusion RRF, puis un reranking par `Qwen3-Reranker-4B`, en BF16 et à révisions de modèle épinglées. L'index, rangé lui aussi dans `.cache/rag/`, contient des preuves *typées* : résumés, sections, tableaux et figures du texte original, mais aussi fiches terminées et lignes de `benchmarks.csv`, chacune marquée comme

transcription ou note. Les résultats en tiennent compte : une fiche ou une ligne de benchmark sert à s'orienter, et l'agent doit remonter au passage original (`rag evidence ID --parent`) avant de citer. `rag benchmarks` filtre les chiffres sans plafond de résultats et conserve `n/r`, les unités et la portée de la puissance ; `rag check-answer` vérifie, pour une réponse enregistrée en JSON, que chaque extrait cité figure dans la preuve invoquée. La skill `/paper-research` encadre l'usage de ce moteur comme `/paper-ask` celui du premier.

Acquisition. Quand le corpus ne suffit pas, `rag acquire` cherche des textes intégraux en accès ouvert (OpenAlex, arXiv), les télécharge dans un répertoire privé, les extrait et les déduplique par DOI, identifiant arXiv et empreinte du document. Rien n'entre dans la base à ce stade : l'agent présente chaque article retenu (titre, année, extrait) et n'appelle `rag approve` que pour ceux que l'utilisateur accepte. Un article importé reçoit une fiche `todo` et n'écrase jamais un article déjà présent. Un budget borne chaque session : 8 recherches, 8 PDF, 50 Mo par PDF, 200 Mo et 15 minutes au total. Un PDF indisponible est traité comme une limite de la recherche, pas comme une preuve d'absence.

Mesure. Les deux moteurs ont été comparés sur les jeux de questions de la section 6.4, avec les mêmes attentes et les mêmes métriques (tableau 5). Cette comparaison date du 26/09/2026, sur le corpus de 28 articles : elle n'a pas été refaite sur 45 articles, faute de GPU sur la machine de la seconde mesure.

Table 5 – Moteur CPU (bge-small) contre moteur GPU (Qwen3), mêmes jeux de questions, corpus de 28 articles. En gras : meilleure valeur par jeu.

Jeu	Moteur	recall@5	recall@10	MRR	nDCG@10	s / question
dev (20)	bge-small hybride + reranking	**1,00**	**1,00**	**0,85**	**0,46**	∼1,4
	Qwen hybride	0,90	0,90	0,78	0,37	0,7
	Qwen hybride + reranking	0,80	0,80	0,76	0,35	3,7
holdout (25)	bge-small hybride + reranking	0,80	0,80	**0,78**	**0,41**	∼1,4
	Qwen hybride + reranking	**0,88**	**0,92**	0,75	0,35	3,3
FR (8)	bge-small dense (meilleur mode)	0,62	0,75	0,46	0,19	0,1
	Qwen dense	**1,00**	**1,00**	0,63	0,22	0,7
	Qwen hybride + reranking	**1,00**	**1,00**	**0,88**	**0,31**	3,8

Le seul écart net est le français : Qwen retrouve les 8 questions sans reformulation en anglais. Sur le holdout, il gagne 2 questions sur 25 au recall@5, à MRR comparable. Sur le jeu dev, il fait moins bien, ce qui est en partie attendu puisque ce jeu a servi à régler le premier moteur. Son canal lexical (FTS5) est plus faible que le BM25 du premier moteur (dev 0,80 contre 0,95 ; français 0,00). Le reranker Qwen est aussi le meilleur signal « hors corpus » mesuré (section 6.5). Il a un coût : un GPU de 32 Go, et des modèles rechargés à chaque requête, soit 3 à 4 s par question avec reranking. Le premier moteur reste donc le moteur par défaut ; le second est une aide pour les questions en français et les écarts de vocabulaire.

5 Ce que le logiciel apporte

Économie de tokens. Le tableau 6 compare, pour quatre tâches courantes, la lecture directe des fichiers et le passage par les commandes. Les ordres de grandeur sont relevés sur le corpus actuel (taille des fichiers divisée par quatre).

Vérifiabilité. Chaque élément renvoyé se rejoue : un passage par son identifiant (`show`) et sa page dans le PDF, une citation par `verify`, un chiffre par sa ligne de `bench` et sa section source,

TABLE 6 – Coût en tokens de tâches courantes (ordres de grandeur).

Tâche	Sans la couche de recherche	Avec
Un chiffre sur une plateforme	lire `benchmarks.csv` en entier : ∼15 000	`bench` filtré : 100–400
Trouver où un sujet est traité	`grep` sur les sections, qui échoue sur les synonymes	`search -k 8` : ∼600
Lire la preuve d'une affirmation	ouvrir `fulltext.md` : ∼13 000 (médiane)	`show` sur 1 à 3 passages : 300–900
Vérifier une citation	relire le passage	`verify` : ∼20

un classement par le déterminisme de la fusion. Une absence de résultat est présentée comme un constat sur la base, pas sur la littérature. Les paraphrases, les rapprochements entre articles et les inférences ne se vérifient pas automatiquement : l'agent doit les étiqueter comme déduits.

Au-delà de la recherche. La même base sert à d'autres tâches de la préparation de thèse :

- les références extraites alimentent `external_refs.csv`, qui classe les travaux absents de la base par nombre d'articles du corpus qui les citent : ce sont les prochaines lectures ;
- `pdb discover` (`/paper-discover`) interroge OpenAlex (citations entrantes, recherche par axe ou par lacune) et propose des candidats, ajoutés seulement avec l'accord de l'utilisateur ;
- `/paper-gaps` tient une carte des zones non couvertes (`gaps.md`), chaque entrée étant appuyée par des identifiants d'articles ;
- une application de visualisation locale (graphe 3D du corpus, frise des citations) donne une vue d'ensemble à l'utilisateur ; l'agent ne s'en sert pas ;
- `pdb plot` produit des figures statiques (PNG, PDF ou SVG) à partir de `benchmarks.csv`, du dernier rapport de `pdb eval` et des métadonnées du corpus : débit contre puissance, GOPS/W par précision, ressources FPGA, précision contre débit, qualité de la recherche, années, axes, citations, lacunes. Chaque figure indique en pied sa source et son effectif (par exemple 52 lignes sur 172, les `n/r` étant exclues), et `figures/manifest.json` les recense. La figure 2 en est un exemple ; les portées de puissance hétérogènes y sont distinguées plutôt que mélangées.

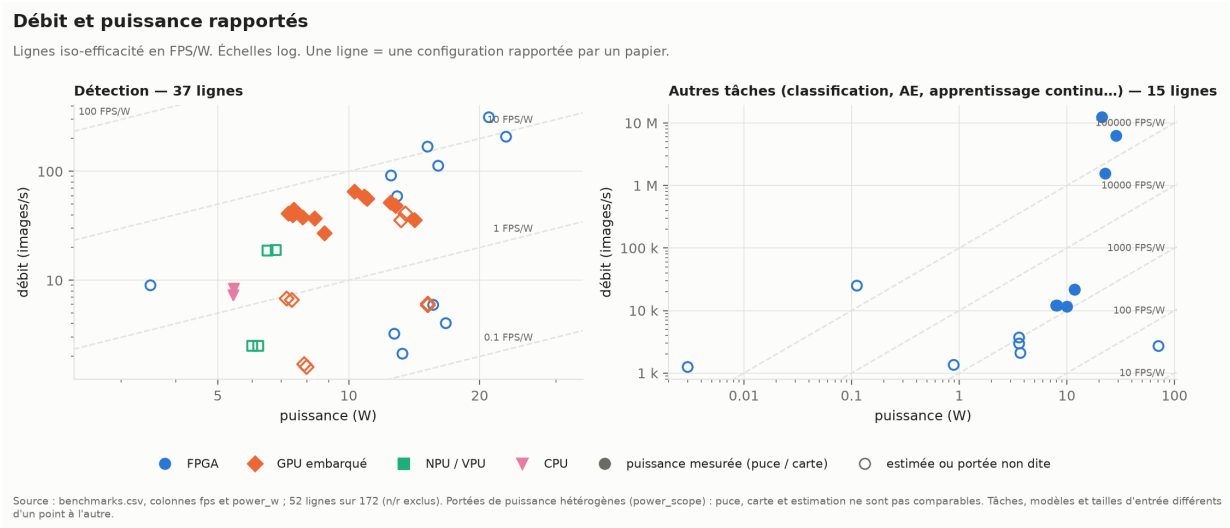


FIGURE 2 – Sortie de `pdb plot bench` : débit et puissance rapportés, une configuration par point (52 lignes de `benchmarks.csv` sur 172). Symboles pleins : puissance mesurée ; creux : estimée ou portée non précisée. Les points ne sont pas comparables entre eux sans lire la portée de la mesure.

État du corpus. Au 26 septembre 2026 : 45 articles, dont 41 avec une fiche terminée, 172 lignes de benchmark, 3 479 passages indexés, et 25 lacunes recensées, dont 10 appuyées par des articles du corpus (sortie de `pdb status`). Le corpus a grossi de 17 articles depuis la première version de ce texte (28 articles, 2 414 passages) ; les fiches et lignes de benchmark des nouveaux articles ont été rédigées par l'agent.

6 Tests et évaluation

6.1 Tests unitaires

La suite (`bin/pdb test`) compte 205 tests répartis en 23 fichiers. Elle s'exécute en une dizaine de secondes, sans GROBID et sans modèle d'embeddings, grâce à des substituts déterministes (tableau 7) :

- une vraie sortie TEI de GROBID (0.8.1), conservée comme fixture, pour le parsing ;
- un encodeur et un cross-encoder factices, fonctions déterministes du texte, pour tester la fusion, le reranking et l'index incrémental sans sentence-transformers ;
- des réponses OpenAlex simulées pour la découverte et l'ajout par DOI ;
- un PDF synthétique produit par PyMuPDF (une légende suivie d'un tableau à filets) pour la reconstruction des tableaux, et des grilles construites à la main pour l'appariement des tableaux Docling ;
- de petits corpus synthétiques (dont une chaîne de citations datée) pour le graphe et la généalogie ;
- des bases et un registre temporaires pour la résolution de la base active, la création d'une base, le scan et l'import entre bases.

Les tests de la visualisation et des figures sont ignorés si leurs dépendances optionnelles ne sont pas installées. Un test fait exception à l'isolement : celui de l'acquisition open access (`test_rag_qwen.py`) charge le tokenizer Qwen pour découper l'article importé, et échoue sur une machine sans réseau où ce tokenizer n'est pas en cache. C'est le seul échec du dernier run (204 tests sur 205) ; le rendre indépendant du modèle est à faire.

Deux garde-fous complètent ces tests à l'usage : `reindex` valide chaque fichier contre le schéma et s'arrête au premier écart, et une liste de contrôles manuels de la chaîne d'ingestion (métadonnées, ordre des sections, absence d'en-têtes résiduels, repli sans GROBID, recoupement des chiffres d'une fiche avec le PDF) est tenue dans la documentation technique.

6.2 Le banc d'évaluation `pdb eval`

Les tests unitaires vérifient que le code fait ce qu'il annonce. Ils ne disent pas si l'extraction est bonne ni si la recherche trouve les bons passages. `bin/pdb eval` le mesure sur le corpus réel, sans LLM ni réseau, et sans rien écrire dans la base. Il couvre quatre volets : l'extraction, la recherche, la vérification des citations et les temps. Un run complet prend quelques minutes sur CPU. `--save` enregistre le rapport en JSON (daté, avec le commit mesuré), et `--compare` signale les écarts avec le dernier rapport, avec un code d'erreur si un seuil de `eval/thresholds.json` n'est plus atteint. La démarche est la suivante : enregistrer avant un changement (extraction, découpage, modèle, fusion), le faire, puis comparer. Le banc porte sur la base active : une base peut avoir ses propres jeux de questions et ses seuils, et ses rapports sont enregistrés dans son dossier (section 6.9). Les chiffres ci-dessous portent sur la base fpga et proviennent d'un run du 26/09/2026 sur 45 articles et 3 479 passages (`bases/fpga/eval/results/2026-09-26T2044-65074b7.json`). Ils ont été reproduits à l'identique le 28/09, après la séparation du code et des données (run `2026-09-28T1039-76dc85e`, aucun écart) ; les temps du tableau 12 viennent de ce second run. Quand c'est utile, les chiffres sont comparés au run précédent, fait le même jour sur 28 articles et 2 414 passages : les jeux de questions n'ont pas changé, seul le corpus a grossi.

TABLE 7 – Couverture des tests unitaires (nombre de fonctions de test par fichier).

Fichier	Tests	Ce qui est vérifié
`test_query.py`	16	filtres, tri et agrégation de `bench`, colonnes de source forcées, identifiants courts, `paper`
`test_tables.py`	14	reconstruction des tableaux (PDF synthétique), filtres et appariement des tableaux Docling
`test_schema.py`	13	validateurs du modèle de données, hiérarchie des titres
`test_tei.py`	12	parsing d'une sortie GROBID réelle
`test_openalex.py`	12	découverte, ajout par DOI et par liste (réponses simulées)
`test_citation.py`	11	exports `.bib`/`.ris` : lecture, appariement, priorité
`test_refs.py`	11	rapprochement des références, `external_refs.csv`, citations internes
`test_rag.py`	12	découpage, BM25, fusion, reranking, index incrémental, `verify` et contrôle des nombres
`test_evaluation.py`	13	métriques du banc sur de petits cas construits : couverture, valeurs de tableaux, rangs, seuils, cas de `verify` ; emplacements du banc par base
`test_plot.py`	10	familles de plateformes, classes de précision, exclusion des `n/r`, rendu de toutes les figures sur la base réelle
`test_lineage.py`	9	généalogie sur une chaîne de citations datée
`test_base.py`	9	base active, `base.json` minimal, règles invalides, classement
`test_xbase.py`	9	scan entre bases, import, création d'une base, onglet Bases
`test_bench.py`	7	banc synthétique : corpus isolé, localisation des preuves, sortie structurée de `claude -p`, contrôle du modèle générateur
`test_graph.py`	8	graphe du corpus sur mini-corpus
`test_ligatures.py`	7	réparation des ligatures perdues
`test_ingest.py`	6	conservation des axes, tags et avertissements à la ré-extraction
`test_viz.py`	6	figures et panneau de détail, sans serveur
`test_status.py`	5	état de la base et prochaines actions
`test_embed.py`	4	cache d'embeddings, branchement dans le graphe
`test_rag_qwen.py`	4	moteur GPU en mode lexical : sources, conditions numériques, instantané périmé, acquisition isolée
`test_reindex.py`	4	notes sur les benchmarks, mesures propres
`test_registry.py`	3	registre des bases, ordre de résolution de la base active

6.3 Qualité de l'extraction

Le banc calcule d'abord des métriques intrinsèques, qui ne demandent aucune annotation. Il réutilise ensuite deux vérités terrain déjà présentes dans la base : les métadonnées vérifiées à la main et les valeurs relevées dans `benchmarks.csv` (tableau 8).

— *La couverture* ne peut pas atteindre 1 : en-têtes, affiliations, texte des figures vectorielles et mots coupés par une césure sont exclus volontairement. Elle se lit en relatif, d'un run ou d'un article à l'autre.

— *L'année et le DOI* sont bas surtout par construction : le PDF arXiv ne porte souvent ni date ni DOI de la version publiée. C'est pour cela que l'ingestion les complète depuis arXiv et depuis les exports de citation. Le banc a aussi révélé un vrai défaut, encore ouvert : un DOI gabarit d'éditeur (MDPI) n'est pas rejeté.

— *Docling* faisait passer, sur 28 articles, la part des tableaux avec corps de 0,66 à 0,90 (41 tableaux reconstruits, avec OCR pour trois articles dont les tableaux sont des images) ; sur 45 articles, 75 tableaux sont reconstruits par Docling et la part atteint 0,88. Il ne changeait ni les valeurs retrouvées ni la recherche : les tableaux cités par les fiches avaient déjà un corps, et les valeurs manquantes viennent de cellules fusionnées par l'extraction, de la prose citée dans la même section source, ou d'une écriture différente. La qualité des tableaux passés par l'OCR est inégale : un contrôle à l'œil de trois tableaux en trouve un propre, un incomplet et un en partie illisible. D'où la mention « à vérifier » et la règle de ne pas en tirer de chiffre sans relecture du PDF.

TABLE 8 – Qualité de l'extraction (45 articles, tous extraits par GROBID ; entre parenthèses, le run sur 28 articles).

Métrique	Valeur
Couverture du texte : part des mots du PDF (bibliographie exclue) retrouvés dans `fulltext.md`	moy. 0,91 ; min. 0,81
Tableaux dont le corps a été extrait (233 tableaux, dont 75 reconstruits par Docling et 4 par PyMuPDF)	0,88 (0,90)
Ligatures restantes, pour 10 000 mots	0,00
Articles dont des sections numérotées n'ont pas été détectées	13 / 45 (7 / 28)
En-tête GROBID seul contre `metadata.json` : titre / année / DOI / arXiv	0,91 / 0,42 / 0,62 / 0,72
En-tête GROBID seul : F1 sur les noms d'auteurs	0,86
Valeurs de `benchmarks.csv` retrouvées dans le tableau cité (588 valeurs)	0,65 (0,74)
idem, lignes qui ne citent que des tableaux (359 valeurs)	0,71 (0,80)
Valeurs de `benchmarks.csv` retrouvées n'importe où dans le texte	0,76 (0,86)

— *Les valeurs retrouvées baissent* (0,74 → 0,65 dans le tableau cité, 0,86 → 0,76 dans le texte). Les 249 valeurs ajoutées viennent des 17 nouveaux articles, dont plusieurs ont des tableaux reconstruits par Docling ou OCR. Le banc ne dit pas si l'écart vient de l'extraction de ces tableaux ou de la transcription faite par l'agent ; les deux restent à contrôler article par article, et c'est ce que ce volet du banc sert à cibler.

6.4 Recherche de passages

Jeux de questions. Chaque question est associée aux sections pertinentes du corpus (identifiées par `article#ordre`, indépendamment du découpage en fenêtres). Ces sections sont choisies par motif textuel dans les articles cibles (par exemple *reuse factor* dans les deux articles hls4ml). Les questions reformulent le sujet sans reprendre le motif quand c'est possible : « removes duplicate bounding boxes » pour la non-maximum suppression. Il y a trois jeux :

— *dev* : 20 questions en anglais, qui ont servi aux choix de conception (modèle, fusion, reranking) ;
— *holdout* : 25 questions en anglais, écrites ensuite et jamais utilisées pour régler quoi que ce soit. Elles couvrent les 12 articles absents du jeu dev et comprennent 2 questions sur des tableaux et 3 questions qui portent sur plusieurs articles. Trois attentes ont été élargies après lecture des passages renvoyés, parce que ces passages répondaient bien à la question (avant cela : recall@5 de 0,68 avec reranking) ;
— *FR* : 8 questions du jeu dev traduites à la main en français.

Métriques. recall@5 et recall@10 (au moins une section attendue parmi les 5 ou 10 premiers passages), MRR (inverse du rang de la première bonne section), nDCG@10 à gain binaire (Järvelin and Kekäläinen, 2002), qui récompense le fait de trouver *plusieurs* sections pertinentes, et paper@5 (au moins un passage d'un article cible parmi les 5 premiers).

Lecture des résultats.

— *Le jeu dev était optimiste.* Sur le holdout, le recall@5 perd 18 à 35 points selon le mode. Le lexical passe de premier à dernier (0,95 → 0,60). Cela confirme le biais attendu : sur le jeu dev, les sections attendues avaient été choisies par mots-clés, ce qui avantageait BM25.
— *Plus d'articles, plus de concurrents.* À questions identiques, passer de 28 à 45 articles ne change presque rien sur le jeu dev, mais fait baisser le holdout dans les trois modes sans reranking : hybride 0,80 → 0,72, dense 0,76 → 0,64, lexical 0,68 → 0,60. Les nouveaux articles apportent des passages proches du sujet qui passent devant les passages attendus. Le reranking absorbe cette concurrence au recall@5 (0,80 dans les deux runs) mais pas

TABLE 9 – Recherche de passages sur 3 479 passages (45 articles, modèle bge-small-en-v1.5), le 26/09/2026. En gras : meilleure valeur par jeu.

Jeu	Mode	recall@5	recall@10	MRR	nDCG@10	paper@5
dev (20)	lexical (BM25)	0,95	0,95	0,75	0,37	**1,00**
	dense	0,90	0,90	0,70	0,36	0,95
	hybride (défaut)	0,90	0,95	0,76	0,39	0,95
	hybride + reranking	**1,00**	**1,00**	**0,85**	**0,44**	**1,00**
holdout (25)	lexical (BM25)	0,60	0,76	0,57	0,27	0,68
	dense	0,64	0,72	0,56	0,32	0,76
	hybride (défaut)	0,72	**0,80**	0,60	0,31	0,80
	hybride + reranking	**0,80**	**0,80**	**0,71**	**0,38**	**0,88**
FR (8)	lexical (BM25)	0,12	0,38	0,06	0,04	0,25
	dense	**0,62**	**0,75**	**0,38**	**0,17**	**0,75**
	hybride (défaut)	0,50	**0,75**	0,37	0,15	**0,75**
	hybride + reranking	0,50	0,50	0,32	0,12	0,62

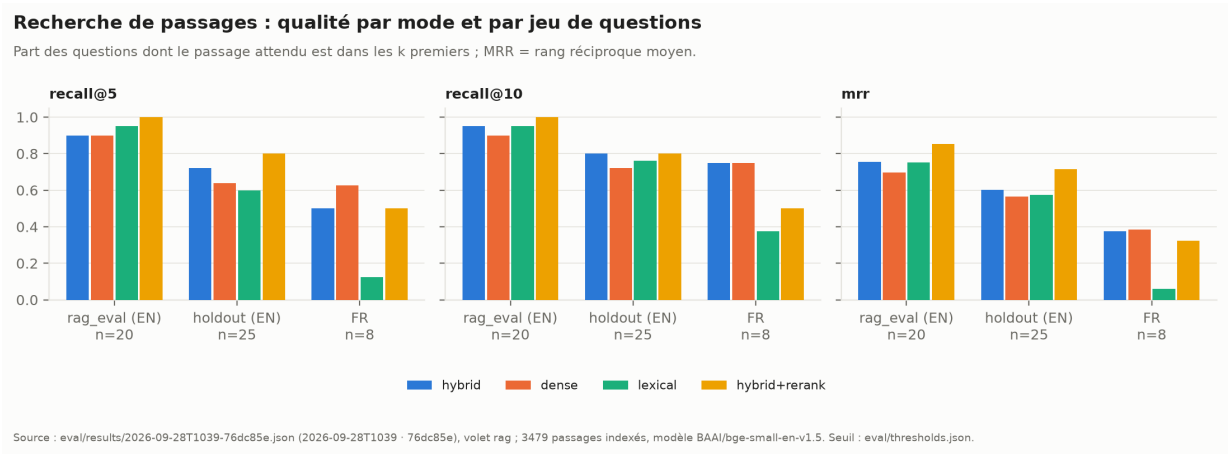


FIGURE 3 – Les chiffres du tableau 9, tracés par `pdb plot eval` à partir du rapport JSON du banc. La ligne pointillée est le seuil d'acceptation (recall@5 $\geq$ 0,8).

entièrement au rang (MRR 0,78 $\rightarrow$ 0,71). Les attentes n'ont pas été revues : un passage d'un nouvel article qui répondrait aussi bien est compté comme un échec.

— *Le reranking devient nécessaire en anglais.* Sur 28 articles, l'hybride seul atteignait le seuil sur le holdout ; sur 45, seul l'hybride avec reranking l'atteint (0,80). Sur le jeu dev, il récupère les deux questions manquées, dont celle sur la suppression des boîtes en double, qui ne contient ni « NMS » ni « non-maximum suppression ».

— *Échecs réels du holdout* (avec reranking) : la tolérance aux radiations, l'entraînement quantifié, la distillation de jeu de données, et les deux questions sur des tableaux (ressources des autoencodeurs d'un article hls4ml, prototypes de FINN). Le texte d'un tableau est court, souvent réduit à sa légende, et se retrouve mal.

— *En français, le lexical s'effondre* (recall@5 de 0,12), et l'index dense résiste mieux (0,62), alors que le modèle n'est pas conçu comme multilingue. Le reranking, entraîné sur MS MARCO en anglais, dégrade le classement. Le moteur GPU (section 4.6) fait nettement mieux sur ce jeu.

— *Décisions.* L'hybride reste le mode par défaut de `pdb search`, parce que le reranking dégrade le français, selon une règle fixée avant la mesure. `/paper-ask` l'active, parce qu'il formule toujours ses requêtes en anglais et que le rang de la première bonne réponse compte quand on n'ouvre que 2 ou 3 passages. Le nouveau run renforce ce choix.

Lors d'une mesure antérieure (25/09/2026, jeu dev), le modèle `all-MiniLM-L6-v2` obtenait un recall@5 de 0,75 en mode dense contre 0,90 pour bge-small, ce qui a motivé le choix de ce dernier. Le seuil d'acceptation retenu est un recall@5 d'au moins 0,8 en anglais : sur 45 articles, le holdout ne l'atteint plus qu'avec le reranking.

6.5 Détecter qu'une question sort du corpus

Peut-on dire, sans LLM, que la base ne traite pas une question ? Deux jeux de 10 questions hors corpus (repliement des protéines, blockchain, etc.) ont été écrits : l'un sert à la calibration, l'autre à la validation. Le second a été écrit avant toute mesure du cross-encoder. Deux signaux sont mesurés : le meilleur cosinus de la recherche dense, et le meilleur score du cross-encoder sur les 3 premiers passages. Pour chacun, le seuil est choisi sur la calibration (jeu dev et premier jeu hors corpus, dont une question à moitié couverte par le corpus a été écartée), en maximisant l'exactitude équilibrée, puis appliqué tel quel à la validation (holdout et second jeu hors corpus). La règle d'adoption a été fixée avant la mesure : en validation, au moins 80 % des questions hors corpus signalées, et au plus 5 % des questions du corpus.

Table 10 – Signal « hors corpus » : part des questions signalées sur la validation (10 questions hors corpus, 25 questions du holdout).

Signal	Seuil	Hors corpus signalées	Corpus signalées à tort	Adopté
cosinus, seuil initial fixé à la main	0,60	0,00	0,00	–
cosinus, seuil calibré	0,78	0,80	0,20	non
cross-encoder, seuil calibré	2,41	0,90	0,40	non
Qwen3-Reranker-4B, 28 articles (section 4.6)	0,96	1,00	0,16	non

Aucun signal ne remplit la règle. Avec bge-small, une question hors sujet obtient un cosinus aussi élevé qu'une question pertinente : le calibrage initial, fait sur quelques requêtes, ne tenait pas, et l'alerte ne se déclenchait jamais. Le cross-encoder sépare mieux les médianes, mais certaines questions du corpus tombent très bas, souvent celles où la recherche échoue déjà. L'alerte a donc été retirée de `pdb search`. Le banc continue de mesurer les deux signaux, pour un essai futur à valider sur un nouveau jeu hors corpus. Le reranker du moteur GPU est le signal le plus proche de la règle : il signale les 10 questions hors corpus, mais aussi 4 des 25 questions du holdout (16 %, pour 5 % tolérés). Il n'est pas adopté non plus.

6.6 Vérification des citations

Le banc tire 40 passages au hasard (graine fixe) et fabrique quatre sortes de citations : une phrase exacte, qui doit être acceptée ; la même phrase avec un mot remplacé, puis avec un chiffre modifié, et une phrase d'un autre article, qui doivent être refusées. « Acceptée » signifie un code de sortie nul (`OK` ou `APPROX` dans le passage indiqué).

Table 11 – Taux d'acceptation de `pdb verify`, avant et après le contrôle des nombres. « Avant » : run sur 28 articles ; « Après » : run sur 45 articles, dont le tirage aléatoire diffère (35 phrases à chiffre modifiable au lieu de 38).

Citation	Avant	Après	Statuts après
exacte (40)	1,00	1,00	40 `OK`
chiffre modifié (35)	**1,00**	**0,00**	35 `NOMBRES`
mot remplacé (40)	0,97	0,88	35 `APPROX`, 3 `NOMBRES`, 2 `ABSENT`
d'un autre article (40)	0,00	0,00	40 `ABSENT`

Avant le contrôle, `verify` acceptait toutes les citations dont un chiffre avait été modifié, alors

que les règles de travail de l'agent comptent sur lui pour garantir les nombres cités. Une première version du contrôle en laissait encore passer 6 (un « 4 » reconnu dans « 6.4 », un « 2 » trouvé ailleurs dans le passage). L'exigence des mots voisins règle les deux cas. Reste non traité : un *mot* remplacé loin de tout nombre donne toujours `APPROX`. C'est pourquoi `/paper-ask` ne garde une citation `APPROX` que présentée comme une paraphrase.

6.7 Temps d'exécution

TABLE 12 – Temps mesurés (médianes de 3 répétitions, CPU sans GPU).

Mesure	Valeur
Extraction hors GROBID, par article (TEI → texte, ligatures, tableaux)	0,01 à 0,14 s
Chargement de l'index (3 479 passages, 7 Mo)	0,29 s
Encodage bge-small	13 passages/s ; index complet ≈ 276 s
`search` à chaud, p50 / p95 : lexical	12 / 24 ms
dense	117 / 166 ms
hybride	126 / 215 ms
hybride + reranking	1 390 / 1 717 ms
`verify` à chaud	246 ms
Commandes à froid, démarrage de Python compris : `status`	0,19 s
`bench`, `paper`	0,09 et 0,08 s
`show`	0,50 s
`search`	6,9 s

Le coût d'une recherche vient presque entièrement du démarrage (import de PyTorch, chargement du modèle) : 6,9 s à froid contre 0,1 s à chaud. C'est le coût réel d'un appel par l'agent, qui lance une commande à chaque fois. Avec 44 % de passages en plus, les temps à chaud restent du même ordre, sauf `verify` (4 → 246 ms). Cet écart n'a pas encore été analysé ; il reste faible devant le coût d'une commande lancée à froid.

6.8 Essai réel de bout en bout

Question posée à `/paper-ask` : « comment les papiers mesurent-ils la puissance, et est-ce comparable ? ». L'agent a enchaîné une requête `bench` (57 lignes, 8 articles), une recherche de 10 passages, la lecture de 3 d'entre eux, 7 vérifications de citations et 8 requêtes d'identité. L'ensemble représente environ 9 Ko lus, soit environ 2 500 tokens, sans ouvrir de texte intégral. Les 7 citations textuelles sont revenues `OK` au premier essai. La recherche a retrouvé en une requête les quatre protocoles de mesure du corpus (PMBus et prise murale, multimètre à l'entrée de la carte, `tegrastats`, puissance au repos mesurée avant les tests). La structure Lu / Déduit / Non trouvé a fait ressortir deux points à vérifier dans la base elle-même, dont un article qui décrit une mesure soignée sans qu'aucune ligne de puissance n'ait été saisie.

L'essai a aussi révélé deux défauts, corrigés depuis et couverts par un test chacun : un identifiant court tronqué pour un nom d'auteur composé, et une agrégation qui demandait un script externe (d'où `bench --count`).

6.9 Un second corpus : la base drone

La base drone (section 3.3) a ses propres jeux de questions, écrits selon le même protocole sur ses 73 articles : 20 questions de développement, 20 réservées, 8 en français et deux jeux de 10 questions hors corpus. Les seuils sont ceux de la base fpga, fixés avant le premier run. Le tableau 13 compare les deux bases (runs du 28/09/2026 au commit 76dc85e).

Tous les seuils sont atteints. La lecture demande de la prudence :

TABLE 13 – Banc d'évaluation sur les deux bases (recall@5 sauf mention).

Mesure	fpga (45 articles)	drone (73 articles)
Couverture du texte : moyenne / minimum	0,91 / 0,81	0,90 / 0,70
Tableaux avec corps	0,88	0,71
En-tête : titre / année / DOI	0,91 / 0,42 / 0,62	0,92 / 0,64 / 0,70
Valeurs de `benchmarks.csv` retrouvées dans le tableau cité	0,65 (588)	0,60 (716)
Dev, hybride / hybride + reranking	0,90 / 1,00	0,95 / 1,00
Réservé, hybride / hybride + reranking	0,72 / 0,80 (25 q.)	0,95 / 0,95 (20 q.)
Réservé, MRR hybride + reranking	0,71	0,92
Français, hybride + reranking	0,50	0,62
Pertinence faible (`cos`) : hors corpus / corpus signalés	0,80 / 0,20	1,00 / 0,10
`verify` : exacte / mot remplacé / chiffre modifié / autre article	1,00 / 0,88 / 0,00 / 0,00	1,00 / 0,85 / 0,00 / 0,00

— **Le jeu réservé drone est plus facile**, et l'on ne peut pas en conclure que la recherche y marche mieux. Le corpus drone est plus hétérogène (détection, suivi, neuromorphique, course de drones) : un fait distinctif y a moins de voisins proches que dans un corpus centré sur les accélérateurs. Deux attentes ont en outre été élargies après lecture des passages renvoyés ; sans elles, le reranking obtient 0,85. Sur 20 questions, 0,05 correspond à une question.
— **Les échecs sont de même nature** que sur la base fpga : un tableau (puissance au repos de plusieurs cartes) manqué par tous les modes, et 3 questions françaises sur 8.
— **La pertinence faible reste inutilisable** : le cosinus signale toutes les questions hors corpus non ambiguës de validation, mais aussi 2 des 20 questions réservées, au-delà des 5 % de fausses alertes tolérés.
— **L'extraction des tableaux est plus faible** (71 % avec corps contre 88 %). La cause est connue : le repli Docling n'a pas été lancé sur cette base. Sur les 41 articles qui gardent un tableau sans corps, 40 ne sont jamais passés par Docling (5 sur 14 pour fpga), et seuls 10 tableaux y ont été reconstruits par Docling, contre 75. C'est une action à faire, pas une limite de l'outil.

Le comportement de `verify` est le même sur les deux corpus : il accepte toutes les citations exactes et refuse tous les chiffres modifiés et toutes les citations d'un autre article.

6.10 Banc synthétique (`pdb synth`)

Les jeux précédents sont petits et écrits par l'auteur du système. `bin/pdb synth` prépare une évaluation plus large, séparée de la base de travail (`.cache/bench/`) :

— *corpus gelé* de 200 articles, avec des sujets voisins servant de distracteurs ; les passages d'ancrage des questions sont filtrés sur le FPGA, l'IA embarquée et l'accélération matérielle, en excluant introductions et généralités ;
— *génération locale* des questions par un modèle ouvert (Qwen3.8-27B quantifié en Q6, servi par llama.cpp sur la même RTX 5090), en JSON contraint par un schéma, avec graine fixe ; l'identité du modèle servi et chaque citation sont contrôlées ;
— *validation sans annotation humaine* : deux sessions indépendantes d'un LLM (Codex pour les 200 questions de référence, puis Claude Code en mode non interactif, `claude -p`) contrôlent chaque question ;
— *quotas* : 40 questions à un passage, 40 numériques, 50 inter-articles, 30 intra-article, 20 sur des tableaux et 20 sans réponse dans le corpus ; partage 40 % développement, 60 % test, sur des ensembles d'articles disjoints, équilibré entre anglais et français, et 40 questions déclinées en variantes traduites et abrégées ;

— *ablations* : les quatre modes de recherche du moteur GPU et les deux modes CPU de `pdb search` sur les mêmes questions, puis des variantes d'agent exécutées dans un bac à sable (bubblewrap, outils de lecture seuls, corpus en lecture seule), notées par un juge LLM.

Les fiches et transcriptions numériques produites automatiquement forment un corpus *enrichi* distinct, qui ne sert jamais à écrire les réponses de référence ; une ligne de benchmark n'y est gardée que si chacun de ses nombres figure dans la section citée. Le protocole et ses tests sont en place ; ses résultats ne sont pas encore disponibles et ne sont donc pas rapportés ici.

6.11 Menaces à la validité

— Les jeux sont petits : sur 25 questions, un écart de 0,04 correspond à une seule question. Le holdout n'a servi à régler aucun paramètre, mais il a été écrit, comme le jeu dev, par l'auteur du système. Des questions écrites par un tiers sont nécessaires pour confirmer les écarts. Le second corpus (section 6.9) montre que les tendances ne tiennent pas à un seul corpus, mais ses questions ont été écrites par le même auteur.
— Les sections attendues sont choisies par motif textuel. C'est une approximation : une bonne réponse formulée autrement est comptée comme un échec (trois attentes du holdout ont dû être élargies pour cette raison).
— La vérité terrain des tableaux ne couvre que les valeurs reprises dans `benchmarks.csv`.
— La qualité des réponses rédigées par l'agent n'est pas évaluée : l'essai de bout en bout est unique, et il a été jugé par l'agent qui l'a exécuté. Une évaluation indépendante demanderait un jugement humain.
— La comparaison des deux moteurs porte sur le corpus de 28 articles ; elle n'a pas été refaite après l'ajout de 17 articles, qui a fait baisser le moteur CPU.
— Le banc synthétique aura ses propres biais : questions générées par un LLM, validées et notées par d'autres LLM. Il mesurera ce corpus gelé et cette distribution de tâches, pas le travail de recherche en général.

7 Limites et perspectives

— **Tableaux.** Ils restent la partie la plus fragile de l'extraction. Docling porte à 90 % la part des tableaux avec corps, mais les tableaux passés par l'OCR sont de qualité inégale et restent marqués « à vérifier ». Les tableaux se retrouvent mal par la recherche, et un chiffre absent de `benchmarks.csv` et pris dans une table mal extraite ne sera pas retrouvé de façon fiable : `bench` reste la référence pour les chiffres.
— **Questions hors corpus.** `search` renvoie toujours des passages, et aucun score testé ne dit de façon fiable que la base ne traite pas le sujet. Ce jugement revient à l'agent. Le reranker Qwen s'approche de la règle (16 % de fausses alertes pour 5 % tolérés). Pistes : combiner le cosinus et le cross-encoder, puis valider sur un nouveau jeu hors corpus.
— **Vérification.** `verify` garantit les nombres, mais un mot remplacé loin de tout nombre passe encore pour une paraphrase (`APPROX`).
— **Métadonnées.** Un DOI gabarit d'éditeur (MDPI) n'est pas encore rejeté à l'extraction ; il est corrigé à la main dans la base, mais un nouvel article du même éditeur en hériterait.
— **Langue.** Corpus et modèle par défaut sont en anglais, et la recherche sur CPU perd beaucoup en français. Le moteur GPU (Qwen3) retrouve les 8 questions françaises, mais au prix d'un GPU de 32 Go. Piste : un petit modèle multilingue sur CPU, à comparer avec la même évaluation.
— **Passage à l'échelle.** De 28 à 45 articles, le recall@5 du holdout en hybride passe de 0,80 à 0,72 ; seul le reranking tient le seuil. La tendance est à suivre à chaque croissance du corpus, avec `pdb eval --compare`, et les attentes des jeux de questions devront être revues quand un nouvel article répond aussi bien.
— **Tests.** Un test de l'acquisition dépend encore du tokenizer Qwen en cache.

— **Latence.** Environ 7 s par `search` à froid, contre 0,1 s à chaud, dues pour l'essentiel à l'import de PyTorch et au chargement du modèle ; `show` et `verify`, qui s'en passent, répondent en moins de 0,5 s. Un processus résident, par exemple un serveur MCP exposant les mêmes commandes, ne paierait ce coût qu'une fois, et permettrait d'utiliser la base depuis d'autres applications. Le même argument vaut pour le moteur GPU, qui recharge ses modèles à chaque requête (3 à 4 s).
— **Généricité.** Le même code sert deux bases (section 3.3), et une nouvelle se crée par `pdb base new` ; ses axes, ses règles et son périmètre se déclarent dans `base.json` et `profile.md`. Restent communs à toutes les bases, et donc marqués par le domaine d'origine : les colonnes de `benchmarks.csv` (FPS, GOPS, ressources FPGA) et les raccourcis de filtre de `bench`. Un sujet sans mesures de performance matérielle demanderait un schéma de benchmark configurable par base.

8 Conclusion

PaperDatabase montre qu'un corpus de recherche de taille modeste peut être rendu exploitable par un agent LLM sans base de données ni service externe : des fichiers texte bien structurés, un tableau de chiffres sourcés et une recherche hybride déterministe suffisent. La génération reste à l'agent, mais tout ce qu'il cite peut être rejoué et vérifié par une commande. Le coût d'une réponse sourcée baisse d'un ordre de grandeur par rapport à la lecture des textes intégraux. Le banc d'évaluation a aussi montré l'intérêt de mesurer ce que l'on croit acquis : il a fait apparaître un jeu de questions optimiste, une alerte qui ne se déclenchait jamais et une vérification qui laissait passer des chiffres faux. Les deux derniers défauts ont conduit à retirer l'alerte et à corriger la vérification. En remesurant après l'ajout de 17 articles, il a aussi montré que la recherche sans reranking perd du rappel quand le corpus grossit. Un second moteur sur GPU règle l'essentiel du problème du français, sans remplacer le premier. Le même code sert enfin une seconde base, isolée de la première, sur laquelle le banc retrouve les mêmes forces et les mêmes faiblesses. Les prochaines étapes sont les résultats du banc synthétique, des questions écrites par un tiers, une meilleure recherche dans les tableaux, la détection des questions hors corpus et un moteur résident qui évite de recharger les modèles à chaque requête.

Références

Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, et al. Docling technical report. arXiv :2408.09869, 2024. Preprint.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi : 10.1145/1571941.1572114.

GROBID contributors. GROBID : machine learning software for extracting information from scholarly documents. `https://github.com/kermitt2/grobid`, 2008–2026. Logiciel, version 0.8.1 utilisée.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4) :422–446, 2002. doi : 10.1145/582415.582418.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO : A human generated MAchine reading COmprehension dataset. In *Workshop on Cognitive Computation (CoCo@NIPS)*, 2016.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv :1901.04085, 2019. Preprint.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex : A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv :2205.01833, 2022. Preprint.

Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi : 10.18653/v1/D19-1410.

Stephen E. Robertson and Hugo Zaragoza. The probabilistic relevance framework : BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4) :333–389, 2009. doi : 10.1561/1500000019.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM : Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack : Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi : 10.1145/3626772.3657878. Preprint arXiv :2309.07597 (2023).

A Glossaire des acronymes

Acronyme	Signification
API	*Application Programming Interface*, interface de programmation
BF16	*bfloat16*, format flottant sur 16 bits
BM25	*Best Matching 25*, fonction de score de la recherche lexicale
BRAM	*Block RAM*, mémoire embarquée d'un FPGA
CPU	*Central Processing Unit*, processeur généraliste
CSV / TSV	*Comma- / Tab-Separated Values*, tableau texte à séparateur virgule / tabulation
DOI	*Digital Object Identifier*, identifiant pérenne d'une publication
DSP	*Digital Signal Processing* : bloc de calcul (multiplieur-accumulateur) d'un FPGA
FPGA	*Field-Programmable Gate Array*, circuit logique reprogrammable
FPS	*Frames Per Second*, images par seconde
FTS5	*Full-Text Search* version 5, index plein texte de SQLite
GOPS	*Giga Operations Per Second*, milliards d'opérations par seconde
GPU	*Graphics Processing Unit*, processeur graphique
GROBID	*GeneRation Of BIbliographic Data*, extracteur de structure d'articles PDF
IA	intelligence artificielle
INT8	entiers sur 8 bits (quantification)
JSON	*JavaScript Object Notation*, format de données texte
LLM	*Large Language Model*, grand modèle de langage
LUT	*Look-Up Table*, cellule logique élémentaire d'un FPGA
MCP	*Model Context Protocol*, protocole d'exposition d'outils à un agent LLM
MRR	*Mean Reciprocal Rank*, moyenne de l'inverse du rang de la première bonne réponse
MS MARCO	*Microsoft MAchine Reading COmprehension*, jeu de données de recherche de passages
nDCG	*normalized Discounted Cumulative Gain*, qualité d'un classement
NMS	*Non-Maximum Suppression*, suppression des boîtes de détection en double
OCR	*Optical Character Recognition*, reconnaissance optique de caractères
PDF	*Portable Document Format*
RAG	*Retrieval-Augmented Generation*, génération augmentée par recherche
RRF	*Reciprocal Rank Fusion*, fusion de classements par rangs réciproques
SHA-1	*Secure Hash Algorithm 1*, empreinte d'un texte
TEI	*Text Encoding Initiative*, format XML de sortie de GROBID
YOLO	*You Only Look Once*, famille de détecteurs d'objets

B Lexique

Terme	Sens dans ce document
agent	LLM qui appelle des outils (lecture de fichiers, commandes) pour accomplir une tâche ; ici Claude Code
benchmark (ligne de)	une configuration rapportée par un article (modèle, plateforme, précision, débit, puissance…), avec sa section source, dans `benchmarks.csv`
chunk, passage	fragment de section indexé pour la recherche, identifié par `<article>#<section>.<k>`
cross-encoder	modèle qui note ensemble une requête et un passage ; sert au reranking
embedding	vecteur qui représente le sens d'un texte ; deux textes proches ont des vecteurs proches
fiche	lecture critique d'un article (`fiche.md`) : problème, méthode, résultats, limites, pertinence
holdout	jeu de questions réservé, qui n'a servi à régler aucun paramètre
index dense / lexical	index par embeddings (sens) / par mots exacts (BM25, FTS5)
lacune (gap)	zone que le corpus ne couvre pas, consignée dans `gaps.md`
`n/r`	« non rapporté » : valeur absente de l'article, jamais estimée
portée de puissance	ce que couvre une mesure de puissance (puce, carte, estimation), colonne `power_scope`
preprint	article diffusé sans relecture par les pairs (arXiv)
recall@k	part des questions dont une section attendue figure parmi les k premiers passages
paper@k	part des questions dont un article attendu figure parmi les k premiers passages
reranking	reclassement des premiers résultats par un modèle plus coûteux
skill	procédure écrite que l'agent suit pour une tâche (`/paper-ask`, `/paper-fiche`…)
toolflow	chaîne d'outils qui transforme un réseau de neurones en accélérateur matériel (FINN, hls4ml…)

C Commandes

Commande	Effet
`bin/pdb status`	état de la base et prochaines actions
`bin/pdb add <arxiv | doi | pdf>`	acquisition, ingestion et réindexation
`bin/pdb ingest [--force | --reparse]`	(ré)extraction des PDF de `inbox/`
`bin/pdb ingest --reparse --tables docling [--ocr]`	tableaux sans corps repris par Docling
`bin/pdb reindex`	`INDEX.md`, `external_refs.csv`, index de passages
`bin/pdb bench [filtres]`	filtre `benchmarks.csv`, colonnes de source toujours affichées
`bin/pdb paper <id | mot>...`	identité d'un ou plusieurs articles
`bin/pdb search "<q>"`	passages classés (hybride, `--rerank` en option)
`bin/pdb show <chunk_id>`	texte exact d'un passage
`bin/pdb verify <chunk_id> "<cit.>"`	contrôle d'une citation
`bin/pdb cites | cited-by <id>`	graphe de citations internes
`bin/pdb discover`	candidats à l'ajout via OpenAlex
`bin/pdb plot [bench | eval | corpus | all]`	figures statiques et `figures/manifest.json`
`bin/pdb viz`	visualisation locale du corpus
`bin/pdb rag index | search | evidence | benchmarks`	moteur GPU (Qwen3) : index, recherche, preuves, chiffres
`bin/pdb rag acquire | approve | check-answer`	acquisition open access, import après accord, contrôle d'une réponse
`bin/pdb synth <étape>`	banc synthétique (corpus, génération, validation, ablations, rapport)
`bin/pdb base list | new | register | use`	bases enregistrées, création, enregistrement, base par défaut
`bin/pdb base info | check`	profil et axes d'une base ; effet des règles de `base.json`
`bin/pdb base scan | import <base>:<id>`	articles utiles d'une autre base ; copie d'un article
`bin/pdb --base <nom> <commande>`	agir sur une autre base que celle du dossier courant
`bin/pdb test`	tests unitaires
`bin/pdb eval [--extract | --rag | --speed]`	banc d'évaluation, par volet ou complet
`bin/pdb eval --save | --compare`	enregistre un rapport ; compare au dernier et aux seuils
`/paper-add`, `/paper-fiche`	skills de l'agent : ajout ; fiche et lignes de benchmark
`/paper-ask`, `/paper-gaps`, `/paper-discover`	réponse sourcée ; carte des lacunes ; découverte
`/paper-research`	réponse sourcée avec le moteur GPU et l'acquisition open access
`/base-profile`	proposition d'affinage du profil et des règles de la base

Installation. Python 3.12 et Docker (pour GROBID) :

```
bash bin/setup.sh                                  # .venv et dépendances
.venv/bin/pip install -r requirements-embed.txt    # optionnel : recherche dense
bin/pdb grobid start                               # GROBID sur localhost:8070
bin/pdb add 2507.18174 && bin/pdb test
```